

Отчёт по лабораторной работе №3

Управляющие структуры

Владимир Базлов

Содержание

Цель работы	2
Задание	2
Задание 8	2
Задание 9	3
Задание 10	3
Задание 11	3
Выполнение работы	3
3.1. Подготовка окружения	3
3.1.1. Установка и проверка окружения	3
3.1.2. Проверка работы ядра	4
3.1.3. Работа с зависшим ядром	4
3.2. Изучение управляющих структур	4
3.2.1. Цикл while	4
3.2.2. Цикл while при работе с массивами	4
3.2.3. Цикл for	4
3.2.4. Цикл for по массиву	4
3.2.5. Тернарный оператор	4
3.2.6. Объявление функций	5
3.2.7. Функции sort и sort!	5
3.2.8. Функции высшего порядка	5
3.2.9. Пакет Colors	7
3.3. Задание 8. Функция outer(x, y, operation)	8
3.3.1. Реализация функции	8
3.3.2. Проверка функции	9
3.3.3. Проверка другой операции	9
3.3.4. Матрицы заданной структуры	9
3.4. Задание 9. Решение системы линейных уравнений	11
3.4.1. Формирование матрицы и вектора	11

3.4.2. Решение системы.....	11
3.4.3. Проверка решения	11
3.4.4. Обобщение решения	11
3.5. Задание 10. Матрица M 6×10 и её анализ	13
3.5.1. Создание матрицы	13
3.5.2. Подсчёт элементов, больших N	13
3.5.3. Поиск строк с двумя вхождениями M_val	13
3.5.4. Поиск пар столбцов	14
3.6. Задание 11. Вычисление двойных сумм.....	15
3.6.1. Вычисление сумм циклами	15
3.6.2. Альтернативное вычисление	15
3.7. Завершение работы	16
3.7.1. Сохранение блокнота	16
3.7.2. Проверка воспроизводимости	16
3.7.3. Подготовка иллюстраций	16
Выводы	17

Цель работы

Основная цель работы — освоить применение циклов, функций и сторонних пакетов Julia для решения задач линейной алгебры и работы с матрицами.

Задание

Задание 8

Написать свою функцию, аналогичную функции `outer()` языка R, с интерфейсом `outer(x, y, operation)`. Для операции `*` она должна быть эквивалентна произведению матриц A ($L \times M$) и B ($M \times N$), где элементы результирующей матрицы C имеют вид:

$$C[i, j] = \sum_{k=1}^M A[i, k] \cdot B[k, j].$$

Используя написанную функцию `outer()`, создать матрицы заданной структуры. Решение должно легко обобщаться на случай матриц большей размерности той же структуры.

Задание 9

Решить систему линейных уравнений с 5 неизвестными, рассмотрев матричное уравнение $Ax = y$. Обратить внимание на особый вид матрицы A . Метод должен легко обобщаться на случай большего числа уравнений, где матрица A имеет такую же структуру.

Задание 10

Создать матрицу M размерности 6×10 , элементами которой являются целые числа, выбранные случайным образом с повторениями из совокупности $1, 2, \dots, 10$. Найти:

- число элементов в каждой строке матрицы M , которые больше числа N (например, $N = 4$);
- определить, в каких строках матрицы M число M_{val} (например, $M_{val} = 7$) встречается ровно 2 раза;
- определить все пары столбцов матрицы M , сумма элементов которых больше K (например, $K = 75$).

Задание 11

Вычислить:

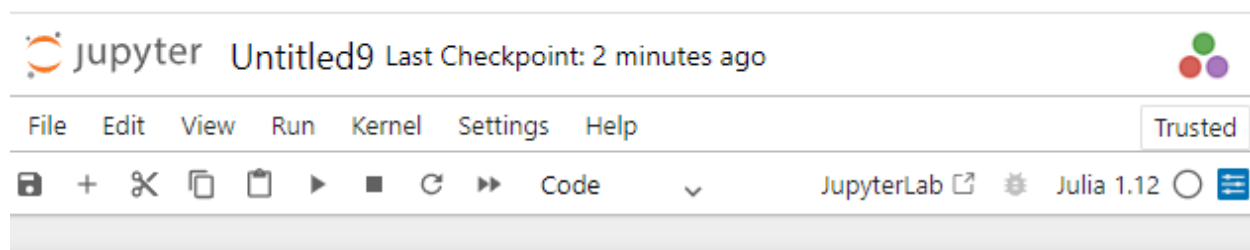
1. $-\sum_{i=1}^{20} \sum_{j=1}^5 \frac{i^4}{3+j},$
2. $-\sum_{i=1}^{20} \sum_{j=1}^5 \frac{i^4}{3+i \cdot j}.$

Выполнение работы

3.1. Подготовка окружения

3.1.1. Установка и проверка окружения

Установлена среда Julia и ядро IJulia для работы в JupyterLab. Создан новый блокнот с ядром Julia.



```
[1]: println("Julia kernel is running")
      1 + 1
      Julia kernel is running
[1]: 2
```

3.1.2. Проверка работы ядра

Выполнена проверка работоспособности ядра: тестовая ячейка отработала, счётчик выполнения сменился с [*] на [1].

3.1.3. Работа с зависшим ядром

При возникновении ситуации с зависшим счётчиком [*] использовались команды Kernel → Interrupt Kernel и Kernel → Restart Kernel для сброса состояния ядра. Ячейки запускались по одной.

Проверка работы ядра

Проверка работы ядра

3.2. Изучение управляющих структур

3.2.1. Цикл while

Изучен цикл while. Написан цикл, выводящий числа от 1 до 10.

Цикл while

Цикл while

3.2.2. Цикл while при работе с массивами

Изучен цикл while при работе с массивами строк: выполнен перебор массива myfriends с выводом приветствия.

Цикл while по массиву myfriends

Цикл while по массиву myfriends

3.2.3. Цикл for

Изучен цикл for. Выполнен перебор по диапазону 1:2:10.

Цикл for по диапазону

Цикл for по диапазону

3.2.4. Цикл for по массиву

Выполнен перебор массива строк myfriends с помощью цикла for.

Цикл for по массиву myfriends

Цикл for по массиву myfriends

3.2.5. Тернарный оператор

Изучен тернарный оператор. Проверено выражение $(x > y) ? x : y$ для $x = 5, y = 10$.

Тернарный оператор

Тернарный оператор

3.2.6. Объявление функций

Изучены три способа объявления функций:

- через конструкцию `function ... end`;
- в однострочной форме;
- с использованием анонимной функции.

Объявление и вызов функций

Объявление и вызов функций

3.2.7. Функции `sort` и `sort!`

Изучено различие между `sort` и `sort!`: первая функция возвращает новый отсортированный массив, а вторая изменяет исходный массив.

Функции `sort` и `sort!`

Функции `sort` и `sort!`

3.2.8. Функции высшего порядка

Изучены функции высшего порядка `map` и `broadcast`, а также точечная запись `f.(...)`.

Функции `map` и `broadcast`

Функции `map` и `broadcast`

```
[3]: n = 0
      while n < 10
        n += 1
        println(n)
      end
```

```
1
2
3
4
5
6
7
8
9
10
```

Ячейка с циклом `while` (вывод чисел от 1 до 10). Слева — номер `[n]`, не `[*]`.

```
[5]: myfriends = ["Ted", "Robyn", "Barney", "Lily", "Marshall"]

i = 1
while i <= length(myfriends)
    friend = myfriends[i]
    println("Hi $friend, it's great to see you!")
    i += 1
end
```

```
Hi Ted, it's great to see you!
Hi Robyn, it's great to see you!
Hi Barney, it's great to see you!
Hi Lily, it's great to see you!
Hi Marshall, it's great to see you!
```

Ячейка с циклом while по массиву myfriends.

```
[7]: for n in 1:2:10
    println(n)
end
```

```
1
3
5
7
9
```

Ячейка с циклом for по диапазону 1:2:10.

```
[9]: myfriends = ["Ted", "Robyn", "Barney", "Lily", "Marshall"]

for friend in myfriends
    println("Hi $friend, it's great to see you!")
end
```

```
Hi Ted, it's great to see you!
Hi Robyn, it's great to see you!
Hi Barney, it's great to see you!
Hi Lily, it's great to see you!
Hi Marshall, it's great to see you!
```

Ячейка с циклом for по массиву myfriends.

```
[11]: x = 5
y = 10
(x > y) ? x : y
```

```
[11]: 10
```

Ячейка с тернарным оператором.

```
[13]: function f1(x)
      x^2
      end

      f2(x) = x^2

      f3 = x -> x^2

      f1(3), f2(3), f3(3)
```

```
[13]: (9, 9, 9)
```

Ячейка с тремя способами объявления функций и их вызовом.

```
[15]: v = [3, 5, 2]
      sort(v)
      println(v)

      sort!(v)
      println(v)
```

```
[3, 5, 2]
```

```
[2, 3, 5]
```

Ячейки `sort(v)` и `sort!(v)`.

```
[17]: map(x -> x^3, [1, 2, 3])

      f(x) = x^2
      f.([1, 2, 3])
```

```
[17]: 3-element Vector{Int64}:
       1
       4
       9
```

Ячейки с `map` и `broadcast`.

3.2.9. Пакет Colors

Установлен и загружен пакет `Colors`. Создана палитра из 100 цветов и матрица `rand(palette, 3, 3)`.

Пакет Colors

Пакет Colors

```
[1]: import Pkg
      Pkg.add("Colors")
      using Colors

      palette = distinguishable_colors(100)
      rand(palette, 3, 3)
```

```
[ Warning: attempting to remove probably stale pidfile
  path = "C:\\Users\\Asus\\.julia\\registries\\.pid"
  @ FileWatching.Pidfile C:\\Users\\Asus\\AppData\\Local\\Programs\\Julia-1.1
  2.0\\share\\julia\\stdlib\\v1.12\\FileWatching\\src\\pidfile.jl:300
  Resolving package versions...
  Project No packages added to or removed from `C:\\Users\\Asus\\.julia
  \\environments\\v1.12\\Project.toml`
  Manifest No packages added to or removed from `C:\\Users\\Asus\\.julia
  \\environments\\v1.12\\Manifest.toml`
```



Ячейка с пакетом Colors: палитра и матрица цветов.

3.3. Задание 8. Функция `outer(x, y, operation)`

3.3.1. Реализация функции

Реализована функция `outer`, принимающая две матрицы и бинарную операцию. Внутри используются вложенные циклы `for` по строкам первой матрицы, столбцам второй и общей размерности k . Для каждой пары (i, j) накапливается сумма

$$\sum operation(x[i, t], y[t, j]).$$

Определение функции `outer`

Определение функции `outer`

3.3.2. Проверка функции

Проведена проверка: для матриц A (2×3) и B (3×2) результат `outer(A, B, *)` совпал с результатом встроенного оператора $A * B$.

Проверка `outer` и матричного умножения

Проверка `outer` и матричного умножения

3.3.3. Проверка другой операции

Дополнительно проверена работа функции с другой операцией: `outer(A, B, +)`.

Проверка `outer` с операцией сложения

Проверка `outer` с операцией сложения

3.3.4. Матрицы заданной структуры

Используя циклы и параметр размерности n , построены матрицы заданной структуры. Решение оформлено как функция от n , что обеспечивает обобщение на большие размерности.

Матрица заданной структуры для $n = 5$

Матрица заданной структуры для $n = 5$

Матрица заданной структуры для $n = 10$

Матрица заданной структуры для $n = 10$

```
[3]: function outer(x, y, operation)
      m = size(x, 1)
      k = size(x, 2)
      n = size(y, 2)

      C = zeros(eltype(x), m, n)
      for i in 1:m
        for j in 1:n
          s = zero(eltype(x))
          for t in 1:k
            s += operation(x[i, t], y[t, j])
          end
          C[i, j] = s
        end
      end
      return C
    end
```

```
[3]: outer (generic function with 1 method)
```

Ячейка с определением функции `outer`. Слева — номер `[n]`, не `[*]`.

```
[5]: A = [1 2 3; 4 5 6]
      B = [1 0; 0 1; 1 1]

      C1 = outer(A, B, *)
      C2 = A * B
      println(C1)
      println(C2)
      println(C1 == C2)

      [4 5; 10 11]
      [4 5; 10 11]
      true
```

Ячейка с проверкой: вывод C1, C2 и true при сравнении C1 == C2.

```
[7]: function structure_matrix(n)
      M = zeros{Int, n, n}
      for i in 1:n
          M[i, i] = 2
          i > 1 && (M[i, i-1] = -1)
          i < n && (M[i, i+1] = -1)
      end
      return M
  end

      structure_matrix(5)
```

```
[7]: 5x5 Matrix{Int64}:
      2  -1   0   0   0
     -1   2  -1   0   0
      0  -1   2  -1   0
      0   0  -1   2  -1
      0   0   0  -1   2
```

Ячейка с матрицей заданной структуры для n = 5.

```
[9]: structure_matrix(10)
```

```
[9]: 10x10 Matrix{Int64}:
      2  -1   0   0   0   0   0   0   0   0
     -1   2  -1   0   0   0   0   0   0   0
      0  -1   2  -1   0   0   0   0   0   0
      0   0  -1   2  -1   0   0   0   0   0
      0   0   0  -1   2  -1   0   0   0   0
      0   0   0   0  -1   2  -1   0   0   0
      0   0   0   0   0  -1   2  -1   0   0
      0   0   0   0   0   0  -1   2  -1   0
      0   0   0   0   0   0   0  -1   2  -1
      0   0   0   0   0   0   0   0  -1   2
```

Ячейка с той же функцией для $n = 10$.

3.4. Задание 9. Решение системы линейных уравнений

3.4.1. Формирование матрицы и вектора

Сформирована матрица A размерности 5×5 особого вида и вектор правой части y .

3.4.2. Решение системы

Система решена матричным методом с помощью оператора `\` из модуля `LinearAlgebra`.

3.4.3. Проверка решения

Выполнена проверка: произведение $A * x$ совпало с вектором y .

Формирование и решение системы

Формирование и решение системы

3.4.4. Обобщение решения

Оформлена функция `solve_system(n)`, строящая матрицу той же структуры для произвольного n и решающая систему. Проведена проверка на $n = 10$.

Функция `solve_system`

Функция solve_system

```
[11]: using LinearAlgebra

n = 5
A = zeros(Float64, n, n)
for i in 1:n
    A[i, i] = 2.0
    i > 1 && (A[i, i-1] = -1.0)
    i < n && (A[i, i+1] = -1.0)
end

y = ones(Float64, n)
x = A \ y

println("A = "); display(A)
println("y = ", y)
println("x = ", x)
println("A*x = ", A * x)

A =
5x5 Matrix{Float64}:
 2.0 -1.0  0.0  0.0  0.0
-1.0  2.0 -1.0  0.0  0.0
 0.0 -1.0  2.0 -1.0  0.0
 0.0  0.0 -1.0  2.0 -1.0
 0.0  0.0  0.0 -1.0  2.0
y = [1.0, 1.0, 1.0, 1.0, 1.0]
x = [2.5, 3.9999999999999996, 4.499999999999999, 4.0, 2.5]
A*x = [1.0000000000000004, 1.0, 0.9999999999999998, 1.0000000000000009,
1.0]
```

Ячейка с формированием A, y, решением $x = A \setminus y$ и проверкой $A * x$. Видны: матрица A, вектор y, вектор x.

```
[13]: function solve_system(n)
      A = zeros(Float64, n, n)
      for i in 1:n
          A[i, i] = 2.0
          i > 1 && (A[i, i-1] = -1.0)
          i < n && (A[i, i+1] = -1.0)
      end
      y = ones(Float64, n)
      return A \ y
  end

  solve_system(10)
```

```
[13]: 10-element Vector{Float64}:
      4.999999999999999
      8.999999999999998
     11.999999999999998
     14.0
     15.0
    15.000000000000002
     14.0
     12.0
      9.0
      5.0
```

Ячейка с функцией solve_system(n) и её вызовом для n = 10.

3.5. Задание 10. Матрица M 6×10 и её анализ

3.5.1. Создание матрицы

С помощью Random.seed! зафиксирован генератор случайных чисел для воспроизводимости. Создана матрица

$$M = rand(1:10, 6, 10).$$

Создание матрицы M

Создание матрицы M

3.5.2. Подсчёт элементов, больших N

Для каждой строки подсчитано число элементов, больших $N = 4$, с помощью count(>(N), M[i, :]).

Количество элементов больше N

Количество элементов больше N

3.5.3. Поиск строк с двумя вхождениями M_val

Найдены строки, в которых число $M_{val} = 7$ встречается ровно 2 раза.

Поиск строк с двумя вхождениями 7

Поиск строк с двумя вхождениями 7

3.5.4. Поиск пар столбцов

Перебором всех пар столбцов (j_1, j_2) при $j_2 > j_1$ найдены пары, сумма элементов которых больше $K = 75$.

Пары столбцов с суммой больше K

Пары столбцов с суммой больше K

```
[15]: using Random
      Random.seed!(42)

      M = rand(1:10, 6, 10)
      display(M)

      6x10 Matrix{Int64}:
      7 7 4 5 4 4 7 9 8 4
      5 7 3 5 4 2 5 2 7 2
      5 5 6 1 5 8 8 3 3 1
      8 3 1 6 9 4 2 10 4 9
      7 7 3 2 4 1 6 4 6 10
      2 7 2 7 6 1 5 8 7 3
```

Ячейка с созданием матрицы M (вывод матрицы 6×10).

```
[17]: N = 4
      counts = [count(>(N), M[i, :]) for i in 1:size(M, 1)]
      println("Число элементов > $N по строкам: ", counts)

      Число элементов > 4 по строкам: [6, 5, 6, 5, 5, 6]
```

Ячейка с подсчётом числа элементов > N по строкам.

```
[19]: M_val = 7
      rows_two = [i for i in 1:size(M, 1) if count(==(M_val), M[i, :]) == 2]
      println("Строки, где $M_val встречается ровно 2 раза: ", rows_two)

      Строки, где 7 встречается ровно 2 раза: [2, 5]
```

Ячейка с поиском строк, где M_val = 7 встречается ровно 2 раза.

```
[21]: K = 75
ncols = size(M, 2)
pairs = Tuple{Int,Int}[]
for j1 in 1:ncols
    for j2 in (j1+1):ncols
        if sum(M[:, j1]) + sum(M[:, j2]) > K
            push!(pairs, (j1, j2))
        end
    end
end
println("Пары столбцов с суммой > $K: ", pairs)

Пары столбцов с суммой > 75: Tuple{Int64, Int64}[]
```

Ячейка с поиском пар столбцов, сумма которых > K.

3.6. Задание 11. Вычисление двойных сумм

3.6.1. Вычисление сумм циклами

Реализованы двойные циклы `for i in 1:20, j in 1:5` с накоплением суммы в переменных S1 и S2:

- $S1 = \sum \frac{i^4}{3+j};$
- $S2 = \sum \frac{i^4}{3+i \cdot j}.$

Вычисление первой суммы

Вычисление первой суммы

Вычисление второй суммы

Вычисление второй суммы

3.6.2. Альтернативное вычисление

Те же суммы вычислены альтернативно через генераторы и функцию `sum`. Результаты совпали.

Вычисление сумм через `sum`

Вычисление сумм через `sum`

```
[23]: S1 = 0.0
      for i in 1:20
        for j in 1:5
          S1 += i^4 / (3 + j)
        end
      end
      println("S1 = ", S1)
```

S1 = 639215.2833333334

Ячейка с вычислением S1 (циклы for).

```
[25]: S2 = 0.0
      for i in 1:20
        for j in 1:5
          S2 += i^4 / (3 + i*j)
        end
      end
      println("S2 = ", S2)
```

S2 = 89912.02146097136

Ячейка с вычислением S2 (циклы for).

```
[27]: S1_alt = sum(i^4 / (3 + j) for i in 1:20, j in 1:5)
      S2_alt = sum(i^4 / (3 + i*j) for i in 1:20, j in 1:5)
      println(S1_alt, " ", S2_alt)
```

639215.2833333338 89912.02146097131

Ячейка с альтернативным вычислением через sum(... for i in ..., j in ...).

3.7. Завершение работы

3.7.1. Сохранение блокнота

Сохранён блокнот JupyterLab со всеми выполненными ячейками.

3.7.2. Проверка воспроизводимости

Выполнен перезапуск ядра (Kernel → Restart Kernel) и последовательный прогон всех ячеек для проверки воспроизводимости результатов.

3.7.3. Подготовка иллюстраций

Сделаны скриншоты всех ячеек с кодом и результатами.

Выводы

В ходе лабораторной работы:

1. Изучены управляющие структуры языка Julia: циклы `while` и `for`, тернарный оператор, условные выражения.
2. Освоены три способа объявления функций, включая анонимные функции и функции высшего порядка.
3. Изучены функции `map` и `broadcast`, позволяющие применять функцию к каждому элементу коллекции без явных циклов.
4. Освоена работа со сторонними пакетами Julia на примере `Colors`: установка через `Pkg.add`, загрузка через `using`.
5. Реализована собственная функция `outer`, аналогичная R-функции `outer()`. Показана её эквивалентность матричному умножению при операции `*`.
6. Освоено решение систем линейных уравнений матричным методом с помощью оператора `\`. Показана обобщаемость решения на произвольную размерность.
7. Освоены приёмы статистического анализа матриц: подсчёт элементов по строкам, поиск вхождений, анализ сумм по столбцам.
8. Освоено вычисление двойных сумм двумя способами — циклами и генераторами.
9. В ходе работы выявлена особенность JupyterLab: при одновременном запуске нескольких ячеек счётчик выполнения отображается как `[*]`, что означает незавершённое вычисление. Задача решена прерыванием ядра и последовательным запуском ячеек.